

Lab 2

DFT, FFT, and Signal Reconstruction

Submission Format:

Each student must submit a single ZIP file named in the following format:

`StudentName_Lab2_StudentID.zip`

The ZIP file should contain:

1. A PDF report named: `Report_StudentName_Lab2_StudentID.pdf`
2. Code files corresponding to each problem, named in the format:
`ProblemX` (where X represents the problem number).

Submission Contents:

- The PDF report should include experiment results with corresponding plots/images.
- Each code file should contain:
 - The complete solution for the corresponding problem.
 - Properly structured and commented in English for clarity.

Submission Email:

Send your submission to: `dsp_speit2026@163.com`

Deadline:

May 29th, 12:00 PM (Noon)

In this lab, you will explore **Discrete Fourier Transform (DFT)**, **Fast Fourier Transform (FFT)**, and **Signal Reconstruction** to optimize your audio processing system.

Your tasks include:

- Computing the DFT of an audio signal and analyzing its spectrum
 - Using FFT for efficient computation
 - Studying the impact of modifying signals in the frequency domain
 - Using Fourier Transform for noise reduction and signal reconstruction
-

Part 1: DFT - Frequency Analysis of Audio Signals

In music production and audio signal processing, we analyze the frequency spectrum to enhance audio, reduce noise, or modify sound effects. Understanding the frequency components of an instrument or voice helps optimize sound mixing.

Problem 1: Compute the DFT of an Audio Signal

Consider the following short audio signal:

$$x[n] = [1, 2, 3, 4, 3, 2, 1, 0].$$

Compute its Discrete Fourier Transform (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N},$$

where $N = 8$.

1. Compute the magnitude spectrum $|X[k]|$ and phase spectrum $\angle X[k]$.
2. Plot the original signal and its spectrum (Magnitude & Phase).

Hints:

- Using the DFT definition with loops: `exp(-1j*2*pi*k*n/N)`.
- `abs(X)` and `angle(X)`
Return the magnitude spectrum and phase spectrum, respectively.
Example: `magX = abs(X);` and `phaX = angle(X);`
- `stem(n, x, 'filled')` and `subplot(m, n, p)`
Useful for plotting discrete-time signals and arranging multiple plots in one figure.
Example: `n = 0:length(x)-1; stem(n, x, 'filled');`

Problem 2: Frequency Domain Modifications and Signal Distortion

In real-world applications, signals can be modified directly in the frequency domain and then reconstructed using inverse transformation.

Your task:

1. Set all even-indexed DFT coefficients to zero (keep only odd frequency components).
2. Compute the inverse DFT (IDFT) of the modified spectrum.
3. Compare the original and modified signals (plot their waveforms).
4. Think: How does keeping only odd frequency components affect the time-domain signal? What kind of distortion appears?

Hints:

- `X_mod = X;`
First copy the original spectrum before modifying it.
- **Even-indexed DFT coefficients**
Since the DFT formula uses $k = 0, 1, \dots, N - 1$, the even- k coefficients correspond to MATLAB positions 1, 3, 5, Example: `X_mod(1:2:end) = 0;`
- `stem(...)` or `plot(...)`
Plot the original and reconstructed signals with the same time index for direct comparison.

Part 2: FFT - Improving Computational Efficiency

In audio signal processing and speech recognition, computing DFT can be computationally expensive. To improve efficiency, we use the Fast Fourier Transform (FFT), which reduces complexity to $O(N \log N)$.

Problem 3: Compute FFT and Compare Performance

For the same signal:

$$x[n] = [1, 2, 3, 4, 3, 2, 1, 0].$$

1. Compute the spectrum using both DFT and FFT, and compare their computation times (measure time for $N = 1024$).
2. Plot the FFT spectrum and compare it with the DFT result.
3. Think: What are the advantages of FFT when dealing with large-scale signals?

Hints:

- `x_long = [x zeros(1, 1024-length(x))];`
Pads the signal to length 1024 before timing the transforms.

- `fft(x_long, 1024)`
Efficiently computes the 1024-point FFT spectrum.
- `tic` and `toc`
Measure elapsed computation time. Example:
`tic;`
`X_fft = fft(x_long, 1024);`
`t_fft = toc;`
- **Direct DFT Implementation**
You may implement the DFT with nested loops and preallocation, e.g.
`X_dft = zeros(1, N);`

Problem 4: Effect of Zero-Padding on Spectral Resolution

In practical applications, we can increase spectral resolution by zero-padding (adding zeros to the signal).

Your task:

1. Append 8 zeros to the original signal to increase its length to 16.
2. Compute the FFT and plot the new spectrum.
3. Compare the spectrum before and after zero-padding.
4. Think: How does zero-padding affect the frequency spectrum? Does it alter the original signal information?

Hints:

- `x_pad = [x zeros(1, 8)];`
Appends 8 zeros so the new signal length becomes 16.
- `fft(x, 8)` and `fft(x_pad, 16)`
Compute the spectra before and after zero-padding.
- `abs(X)`
Use the magnitude spectrum to compare the frequency-domain sampling density.

Part 3: Signal Reconstruction - Speech Signal Denoising

In speech processing, signals often contain unwanted noise. Fourier Transform can be used to remove noise and reconstruct the original signal.

Problem 5: Low-Pass Filtering for Speech Signal Reconstruction

Consider a noisy speech signal:

$$x[n] = \sin(0.2\pi n) + 0.5 \sin(0.7\pi n) + 0.2\varepsilon[n],$$

where $\varepsilon[n]$ is random noise with zero mean and variance 1.

Your task:

1. Compute the FFT of the signal and plot its spectrum.
2. Retain only the lowest 10% of frequency components (set high frequencies to zero) and perform inverse FFT (IFFT).
 - If the signal has N frequency components, keep only the first $0.1N$ coefficients.
3. Plot the filtered signal and compare it with the original noise-free signal.
4. Think: Does low-pass filtering completely remove noise? Does it lose important speech information?

Hints:

- `n = 0:N-1;` and `randn(1, N)`
Use these to generate the noisy signal. Example:
`x_clean = sin(0.2*pi*n) + 0.5*sin(0.7*pi*n);`
`x = x_clean + 0.2*randn(1, N);`
- `fft(x)`
Computes the spectrum of the noisy signal.
Example: `X = fft(x);`
- `X_lp = zeros(size(X));`
Creates a low-pass spectrum mask. Example:
`keep = round(0.1*N);`
`X_lp(1:keep) = X(1:keep);`
- `ifft(X_lp)` and `real(...)`
Reconstruct the filtered signal in the time domain.
Example: `y = real(ifft(X_lp));`